

Batch Image Denoising and Edge Detection with OpenMP and CUDA

Gerasimou Dimitrios AEM: 10813 and Stavrianidis Giannis AEM: 11065

Department of Electrical and Computer Engineering, Aristotle University of Thessaloniki
050: Parallel and Distributed Systems

Thessaloniki, Greece
September 2026

Abstract

For this assignment, we present a batch tool that denoises grayscale images with *Non-Local Means* (NLM) and extracts their edges with the *Canny* detector, on multicore CPUs (**OpenMP**) and on NVIDIA GPUs (**CUDA**). The noise level and the filter strength are estimated for every image, and the CPU and the GPU produce identical **deterministic** output. The main techniques are integral images for the patch distances, row **bands** sized for the L2 cache and drawn from one shared pool on the CPU, and a shared-memory tile kernel with a register sliding window, a batch pipeline and a two-slot CUDA stream driver on the GPU. On an Intel i7-11800H and an RTX 3060 Laptop GPU, tested on 36 669 images of four datasets (two chest X-ray collections with added Poisson noise, AAPM low-dose CT, and the real noise of 2DeteCT), denoising runs at 1.1 Mpx/s on one thread, 7.0–7.3 Mpx/s on 16 threads and 143–149 Mpx/s on the GPU: **20–21× faster than the CPU and 128–139× faster than one thread**. The PSNR rises by 4.5–9.2 dB on average, and the edge F1 score on 2DeteCT from 0.25 to 0.75.

1 Introduction

X-ray and CT images are noisy, and the noise breaks the edge detection that often follows. Non-Local Means [1, 2] removes it by averaging every pixel with all pixels whose surroundings look alike, which preserves structure but the direct method costs $(2S+1)^2(2P+1)^2 \approx 11\,025$ operations per pixel with the usual 21×21 search window and 5×5 patches: about 10^{14} operations for the 8.3 gigapixels of one of our datasets. This work parallelizes the pipeline (read, decode, denoise, detect edges, encode, write) for batches of images on the CPU and the GPU, and measures its speed and the quality of its output.

2 Algorithms

Non-Local Means Every output pixel is the weighted mean of the pixels in its search window. The weight of a candidate falls exponentially with the squared distance between the two patches, $w = e^{\frac{-\max(d^2 - 2\sigma^2, 0)}{h^2}}$ with $h = k\sigma$ [2]. For a fixed search offset, the squared differences of all pixels form an image, and its *integral image* (summed-area table [5, 6]) gives every patch sum with four lookups, which removes the patch size from the cost. All sums are exact 32-bit integers, so the result does not depend on the thread count and is the same on the GPU. **Noise level and strength.** σ is estimated per image as the larger of Immerkær’s estimate [4], exact for white noise but too low for the correlated noise of CT reconstructions, and the noise of the flattest 16×16 blocks. The strength $k = \frac{h}{\sigma}$ rises from 0.7 (white noise) to 1.6 (strongly correlated noise) with the ratio of the two estimates. When edge detection follows, it is 0.7 times that for near-white noise, since the edge score peaks at a milder strength than the PSNR.

Canny [3]: Configured using Gaussian blur ($\sigma = 1.4$), Sobel gradient, non-maximum suppression and hysteresis (thresholds 20 and 50), all in integer arithmetic (fixed-point weights, squared magnitudes), so that both backends agree exactly.

Quality is measured by the PSNR against the ground truth, and by the *edge F1*: the Canny edges of the noisy (or denoised) image against those of the ground truth, with a tolerance of one pixel.

3 Parallelization Strategies

Batch pipeline Images are processed in batches of B (256 by default). The stages run one after the other on a batch, each parallel over its images. With the GPU, the CPU threads read, decode and prepare batch $k+1$ and encode and write batch $k-1$ while the GPU works on batch k . On the CPU alone it gains nothing, since decoding and encoding already keep all cores busy (the idle time was 0.3% of a run). With the GPU it hides all CPU work (Section 4).

Bands on the CPU Every image is cut into horizontal **bands** of 16–32 rows, about four per thread over the batch, and the bands of *all* images of the batch, wider images first, form one pool from which the threads draw with `schedule(dynamic)`. With fewer images than threads, each image gets a multiple of the thread count of bands, so a single image keeps every thread busy. This is the only CPU scheduling. Canny uses the same bands.

- **Cache** For one search offset the integral image has one 4-byte entry per pixel: 4 MB for a 1 Mpx slice, rebuilt for each of the 441 offsets. For a band, the integral images of the 4 offsets processed together and the three accumulators per pixel (weight sum, weighted sum, largest weight, 0.4 MB) stay in the core’s L2 cache (1.25 MB) during all 441 offsets, so the offset loop runs from cache, not from memory.

- **Load balance** Our images range from 144×124 to 2916×2863 pixels. With one image per thread every batch waits for its largest image (97.2% of the threads busy on the X-ray set). With small equal-cost bands the idle time at the end of a batch is at most one band (99.9% busy, 15.99 of 16 threads).
- **Independence** A band needs only its own rows of the padded image plus $2P$ margin rows, so there is no synchronization between offsets or threads. The price is the margin: $2P$ of the band + $2P$ difference rows are recomputed. Canny bands recompute the $4 + 2$ -radius blur rows the same way. The thread that finishes the last band of an image runs its hysteresis flood fill at once, while the others continue with other images.

The GPU kernel A block of 32×16 threads computes a 32×32 tile (two pixels per thread). All threads compute the horizontal patch sums of the four offsets straight from the tile, holding a sliding window of squared differences in registers. After one barrier each thread slides a vertical window down its two pixels and accumulates the weights, in the same order as the CPU. The kernel is a C++ template over P (0–10), so the window lives in registers and every division is by a constant. The weights come from the same function `nlm_weight()` as the CPU’s table, built from adds, multiplies and explicit `fmaf`, so the two results are identical (`-fmad=false, -ffp-contract=off`).

Streams Images alternate between two *slots*, each with its own CUDA stream, device buffers and pinned host memory. While an image’s kernels run, the next image is uploaded and the previous one downloaded. **Canny on the GPU** uses one thread per pixel for blur, gradient and suppression. **Hysteresis** is a connectivity problem, like the connected components of the earlier assignments: each 32×32 tile spreads edges in shared memory until nothing changes, and the host relaunches until no tile changes. Only tiles whose neighbours changed a border pixel are *active* in the next launch, and with `-d -e` the denoised image stays on the GPU.

4 Performance Optimizations

Table 1 lists the techniques in order of the measured effect, each measured on the code immediately before it was applied. The bases differ, so the factors are not multiplied. The GPU steps were guided by profiling (Nsight Compute): idle lanes (42% lane use in the difference phase), then shared-memory traffic (MIO throttle, 43% of the stalls) and bank conflicts (45% of the shared-memory cycles were wasted), and on noisy images the weight-table gathers, which missed the L1 cache (12.9% hits against 98.8% on clean images).

#	Technique	Effect
1	NLM on the GPU (all techniques below)	20–21× over 16 threads, 128–139× over 1 thread
2	Integral images for the patch sums, with a weight table (CPU)	5.6×
3	GPU kernel restructuring v1→v4: no integer division, 32-row tiles with 2 pixels per thread, dense element mapping, register window, 4 offsets per barrier, template over P	3.2×
4	Bands and fast interior path for Canny (CPU)	3.0×, threads busy 9.7 → 15.6
5	Table-free weights on the GPU (no table gathers)	1.49×
6	Offset blocking on the CPU: 4 offsets per pass, independent running sums	1.14× at 16 threads, 1.17–1.48× at 1
7	Batch pipeline with the GPU (batches of 16 instead of one batch)	1.24×
8	Chunked rejection: branch-free sums, 16 pixels skipped at once	1.10–1.18×
9	Active tiles in the GPU hysteresis	hysteresis –28%
10	Explicit <code>fmaf</code> in the weight polynomial; conflict-free tile pitch with one 16-byte store	1.06×; 1.04×
11	Band pool with dynamic scheduling (load balance)	threads busy 97.2% → 99.9% (≈3%)
12	Pinned memory with two streams	downloads 3.6× faster, –5% (edges only)

Table 1: Optimization techniques ranked by measured effect. Overall, the CPU version became 10.7× faster than the first direct implementation.

Tried and rejected `-march=native` gave nothing. The loops are limited by the integral image’s dependency chain and by data-dependent branches, not by vector width. Skipping weights below 10^{-3} (as OpenCV does) was 18–24% *slower* on noisy images, since the accept/reject branch becomes unpredictable, and changed the output. Preselecting candidates by local statistics [7, 8] saves little, as a rejection already costs only four lookups. Symmetric weights would break the bit-exact match with the reference.

5 Experimental Setup

System: Intel Core i7-11800H (8 cores, 16 threads, 1.25 MB L2 per core), 15.3 GiB RAM, NVIDIA GeForce RTX 3060 Laptop GPU (compute capability 8.6, 30 multiprocessors, 5.7 GiB), Arch Linux, CUDA 13.4, `gcc -O3 -fopenmp, nvcc -O3`, performance governor, threads bound with `OMP_PROC_BIND=close OMP_PLACES=cores`. **Speed** is measured on a random sample of 500 images of each dataset: a thread sweep on the CPU (batch of 64) and a batch sweep (4–256) on the CPU and the GPU, for denoising and for edge detection, with one warmup and three timed trials (median). The pipeline time covers reading, decoding and filtering (no output). **Quality** is measured on every image, and the complete datasets are also timed end to end (batches of 16, output written, one trial). All runs use the default options. The datasets (36 669 images) are:

- **Chest X-ray (pneumonia):** the 5856 pediatric radiographs of the Kaggle pneumonia set [9]. Poisson noise is added to the originals, which are the ground truth.

- **Chest X-ray (mixed)**: 25 553 radiographs of mixed origin (10 641 tuberculosis, 9088 normal, 5824 pneumonia cases), 144×124 to 2916×2863 pixels. Poisson noise is added as the previous.
- **AAPM Low-Dose CT**: 4260 slices of 512×512 [10]. The quarter-dose slice is the input, the full-dose slice the truth.
- **2DeteCT**: 1000 slices of 1024×1024 [11]. The mode-1 (low dose) reconstruction is the input and the mode-2 (high fidelity) reconstruction the truth. The noise of both is real or simulated on the projections, and spatially correlated.

6 Results

6.1 Speed

Table 2 and Figure 1 summarize the 500-image sweeps. **Denoising costs the same per pixel on every dataset**, whatever the noise level, strength or image size: 1.1 Mpx/s on one thread, 7.0–7.3 Mpx/s on 16 threads, 143–149 Mpx/s on the GPU. The CPU scales 6.4–6.5 $\times$ on 16 threads, and already 6.3–6.5 $\times$ on 8 (80% efficiency). The machine has 8 physical cores, and hyperthreading adds only 0.1–2.7% to denoising, but 14–22% to the edge detection. The GPU is **20–21 $\times$ faster than the best CPU run** for denoising and 2.9–5.7 $\times$ for edges, where decoding the PNG files takes about as long as the filter and limits the gain, most on the smallest images (AAPM). On the GPU, batches of 4–16 images are fastest for denoising and 64 for edges. A batch of 256 is 8–13% slower and needs 3–6 $\times$ the memory of a batch of 16, while the CPU time hardly depends on the batch size above 16.

Batch size Figure 2(a) shows the time of each configuration relative to its best batch size, averaged over the four datasets. Small batches hurt most where the work per image is smallest: GPU edge detection is 2.9 $\times$ slower with 4 images per batch than with 64, CPU edge detection 1.6 $\times$, GPU denoising 17%, while CPU denoising stays within 8%. Each batch pays a fixed cost (stage barriers, filling and draining the pipeline) that fewer images share. Large batches do not make it faster: from 16 to 256 images the memory of the GPU run grows 3–6 $\times$ (Figure 2(b)) and that of the CPU run 3–9 $\times$, and the GPU denoising becomes 10% slower, most likely because the pipeline overlaps only the batches after the first, and 500 images make two batches of 256. A batch of 16–64 is the best compromise.

Dataset	Mpx	Denoising time (s)				Speedup			Edge detection time (s)				Speedup	
		1 thr.	16 thr.	GPU	16/1 thr.	GPU/16	GPU/1		1 thr.	16 thr.	GPU	GPU/16		
Chest X-ray (pneumonia)	710	636.8	100.2	4.97	6.4 $\times$	20.1 $\times$	128 $\times$		21.05	2.68	0.596	4.5 $\times$		
Chest X-ray (mixed)	341	306.3	47.6	2.36	6.4 $\times$	20.2 $\times$	130 $\times$		10.23	1.30	0.332	3.9 $\times$		
AAPM Low-Dose CT	131	116.1	17.9	0.88	6.5 $\times$	20.3 $\times$	132 $\times$		4.21	0.49	0.172	2.9 $\times$		
2DeteCT	524	490.2	75.0	3.52	6.5 $\times$	21.3 $\times$	139 $\times$		18.58	2.27	0.400	5.7 $\times$		

Table 2: Speed on 500 image samples (best batch size of each run, median of 3 trials).

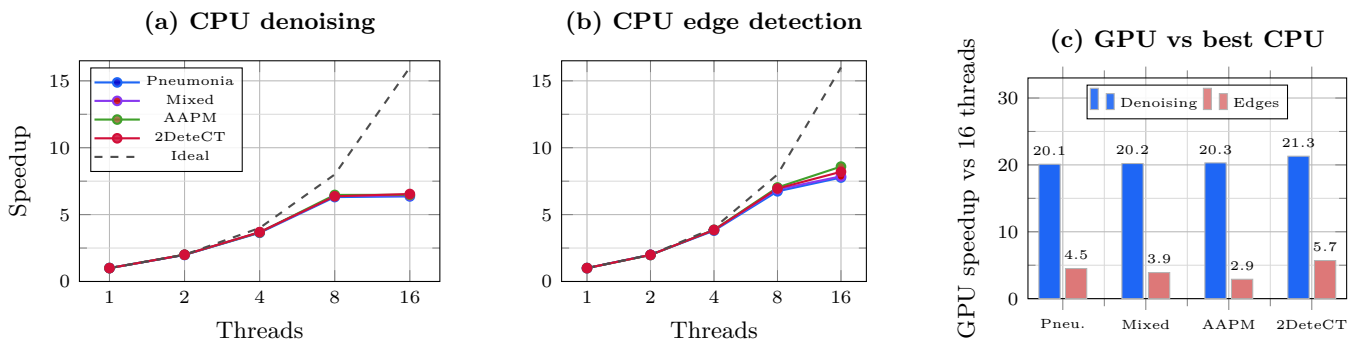


Figure 1: Speedup of the CPU thread sweeps (a, b) and of the GPU over the best CPU run (c), for the four datasets.

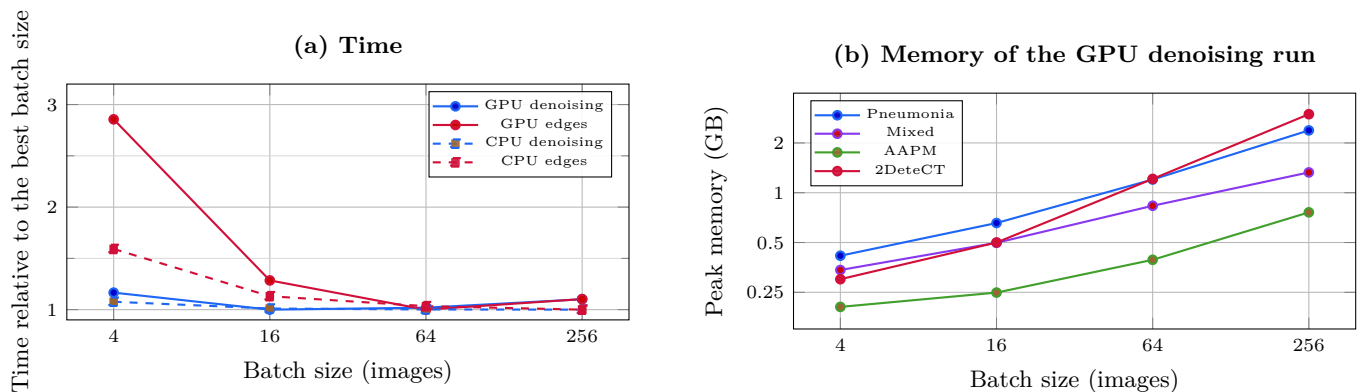


Figure 2: Effect of the batch size (500-image sweeps). (a) Time relative to the best batch size of each configuration, mean of the four datasets. CPU with 16 threads. (b) Peak resident memory of the GPU denoising run, per dataset.

6.2 Complete Datasets

Table 3 gives the end-to-end times of the runs on every image of each dataset: read, decode, filter, encode and write the result, with 16 threads, batches of 16 and one trial. The GPU is 20–22× faster than the CPU here as well (17.6–21.5× when edges follow), although every result image is encoded and written: the batch pipeline hides this CPU-side work behind the kernels, and the whole run reaches 142–150 Mpx/s, as in the 500-image sweeps. The CPU needs 20 minutes for the pneumonia set and 44 minutes for the mixed set, the GPU 58 and 130 seconds. Edge detection after the denoising adds 7–8% on the CPU and 8–10% on the GPU, and 21% on the small AAPM slices, where decoding dominates.

Dataset	Images	Mpx	Denoising: time (s)				Denoising & Canny: time (s)			
			CPU	GPU	Speedup	GPU Mpx/s	CPU	GPU	Speedup	GPU Mpx/s
Chest X-ray (pneumonia)	5 856	8 309	1 186.1	57.7	20.5×	144	1 272.4	62.4	20.4×	133
Chest X-ray (mixed)	25 553	18 486	2 635.2	130.0	20.3×	142	2 835.0	143.1	19.8×	129
AAPM Low-Dose CT	4 260	1 117	153.2	7.7	20.0×	145	164.0	9.3	17.6×	120
2DeteCT	1 000	1 049	151.9	7.0	21.7×	150	163.6	7.6	21.5×	138

Table 3: End-to-end times on the complete datasets (16 threads, batches of 16, output written, one trial).

6.3 Quality

Table 4 gives the quality on every image of each dataset. The noise added to the chest X-rays is estimated 3–4% low (10.4 against 10.9, and 10.7 against 11.0 gray levels). Denoising raises the PSNR by 4.5–9.2 dB on average, and only 4 of the 36 669 images get worse, by at most 1.6 dB (all tuberculosis radiographs, which already have the most reliable noisy edges). The edge F1 rises on every dataset, most where the noisy edges are useless, from 0.25 to 0.75 on 2DeteCT, where every one of the 1000 slices improves (Figure 3). Where the noisy image is already reliable, denoising can lower the score. In the cleanest quarter of the AAPM slices the F1 changes by -0.014 , and in the cleanest quarter of the mixed chest X-rays 35% of the images end with a lower edge score than the noisy input. This is the price of a strength that favours the PSNR. The worst AAPM slice (18.2 dB) gains only 0.2 dB. Structured artifacts are not noise and a denoiser cannot repair them. The 500-image samples agree with the full sets to within 0.13 dB and 0.003 F1.

Dataset	Images	Noise σ added / est.	PSNR (dB)			Edge F1			F1 lower than noisy
			noisy	denoised	gain	noisy	denoised	gain	
Chest X-ray (pneumonia)	5 856	10.9 / 10.4	27.63	36.87	+9.24	0.691	0.800	+0.110	6%
Chest X-ray (mixed)	25 553	11.0 / 10.7	27.51	36.24	+8.74	0.738	0.798	+0.061	22%
AAPM Low-Dose CT	4 260	– / 12.3	26.08	30.57	+4.48	0.821	0.852	+0.031	32%
2DeteCT	1 000	– / 18.6	22.77	28.46	+5.69	0.249	0.749	+0.500	0%

Table 4: Quality on all images of each dataset (means). σ in gray levels. The last column is the share of images whose edge F1 is lower after denoising.

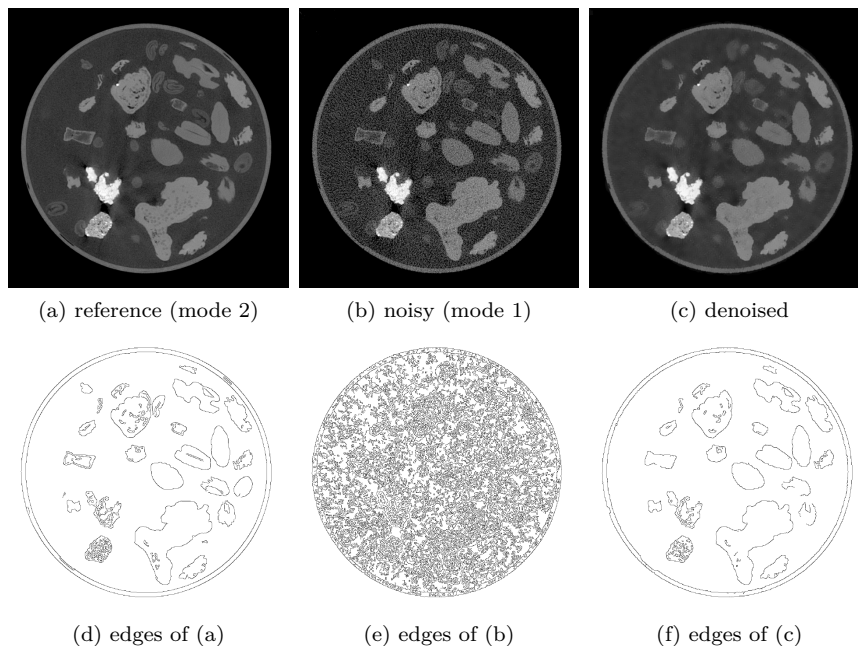


Figure 3: One 2DeteCT slice: PSNR 27.9 $\rightarrow$ 36.6 dB, edge F1 0.27 $\rightarrow$ 0.87. The Canny edges of the noisy slice (e) are dominated by noise, those of the denoised slice (f) follow the reference (d). Gray images shown with a display gamma of 0.55, edge maps inverted (dark edges).

7 Analysis and Discussion

- **The GPU suits NLM:** heavy, regular, independent work per pixel. The whole pipeline reaches ≈ 15 ps per pixel and search offset (the first kernel: 52 ps), which is 143–149 Mpx/s, **edge detection is the opposite**. Cheap per pixel, with decoding and hysteresis (an irregular connectivity problem) dominating, hence only 2.9–5.7 $\times$. The copies that are 2% of the NLM GPU time were 27–42% of the Canny GPU time before the streams.
- **Efficiency is bounded by the hardware:** 80% on 8 cores and 40% on 16 threads. NLM already keeps each core busy, so hyperthreading adds nothing, unlike the memory-bound edge detection (14–22%).
- **Exactness** made every step testable: the CPU, the GPU and an independent NumPy implementation agree bit for bit, so each optimization was checked for identical output.

8 Conclusions

`imgfilter` denoises and edge-detects batches of images on the CPU and the GPU with identical results, and needs no tuning. Bands sized for the L2 cache and drawn from one pool make the CPU version scale as far as the eight cores allow (6.4–6.5 $\times$). A profile-driven tile kernel and a two-slot stream pipeline make the GPU **20–21 $\times$ faster than the 16-thread CPU** for denoising (128–139 $\times$ over one thread). The automatic noise estimate and strength raise the PSNR by 4.5–9.2 dB and the edge F1 on real CT noise from 0.25 to 0.75.

Future work: Decode PNG on the GPU (it limits the edge speedup). Fuse the Canny kernels and propagate whole rows per hysteresis iteration. Choose the strength per image for the edge score, since the cleanest slices lose a little. Symmetric weights with a tolerance based reference. Multi-GPU batches.

Code Availability: Source code, benchmark scripts and all results are available at:

<https://github.com/dimgerasimou/pds-hw4-img-processing>.

References

- [1] A. Buades, B. Coll, J.-M. Morel, A non-local algorithm for image denoising, *CVPR*, 2005.
- [2] A. Buades, B. Coll, J.-M. Morel, Non-local means denoising, *Image Processing On Line* 1, 2011.
- [3] J. Canny, A computational approach to edge detection, *IEEE TPAMI* 8(6), 1986.
- [4] J. Immerkær, Fast noise variance estimation, *Computer Vision and Image Understanding* 64(2), 1996.
- [5] F. C. Crow, Summed-area tables for texture mapping, *SIGGRAPH*, 1984.
- [6] J. Darbon et al., Fast nonlocal filtering applied to electron cryo-microscopy, *IEEE ISBI*, 2008.
- [7] M. Mahmoudi, G. Sapiro, Fast image and video denoising via nonlocal means of similar neighborhoods, *IEEE Signal Processing Letters* 12(12), 2005.
- [8] P. Coupé et al., An optimized blockwise nonlocal means denoising filter for 3-D MR images, *IEEE TMI* 27(4), 2008.
- [9] D. S. Kermany et al., Identifying medical diagnoses and treatable diseases by image-based deep learning, *Cell* 172(5), 2018.
- [10] C. H. McCollough et al., Low-dose CT for the detection and classification of metastatic liver lesions: results of the 2016 Low Dose CT Grand Challenge, *Medical Physics* 44(10), 2017.
- [11] M. B. Kiss et al., 2DeteCT – a large 2D expandable, trainable, experimental computed tomography dataset for machine learning, *Scientific Data* 10, 2023.